

Evidence supplement: TxGraffiti Conjecture 15/3 resolution

Release candidate 4.0.0-rc1

6 August 2026

1. Scope

This supplement records the object identity, proof dependencies, replay architecture, assurance state, environment, provenance and limitations for `txgraffiti-c3-resolution/4.0.0-rc1`.

The canonical mathematical conclusion is

$$\mu^*(G) = 15 < 16 = i(G)$$

for the connected cubic graph encoded in the release. The exact release is an **unrefereed candidate**. All bundled theorem-critical deterministic checks passed in the recorded environment and from a clean extracted copy. Exact-release independent rerun, independent reimplementation, formal verification and conventional peer review remain false.

2. Canonical object and integrity

2.1 Graph files

- `counterexample.json` - canonical formula, graph, matching and witnesses.
SHA-256: 1753ad874d1b09f101f06e91f7a53cfc27026fb4c1166bf7bd16269dae7fbd0f
- `counterexample.edgelist` - raw edge list.
SHA-256: 63363b5e9c8bdd119418024fe80da7bba36f71119f6e89778519d3fbd8a916b2
- `counterexample.g6` - graph6 encoding.
SHA-256: d8b6dc5c5738ac657edfa17193363b48f9416111278e6de07246889668fcb3c6
- `ids_le15.tree.gz` - lower-bound proof tree.
SHA-256: a14c38f6b2f1ed9c65da3efccb60e8443b033b96f8de9cec90fb5d82d020f988

`check_encodings.py` parses all three graph representations, confirms graph identity and checks both explicit independent dominating sets.

2.2 Outer and inner integrity

`MANIFEST.sha256` lists every regular release file except itself. `check_manifest.py` rejects missing, unlisted or changed files. The external `.zip.sha256` sidecar identifies the immutable archive. These are source-supplied integrity records, not independent provenance.

3. Claim-to-evidence ledger

ID	Claim	Principal evidence	Status
C1	Graph is simple, connected, cubic, order 50, size 75, girth 5	Encodings and <code>check_encodings.py</code>	Deterministically checked

ID	Claim	Principal evidence	Status
C2	$\mu^*(G) = 15$	Explicit maximal matching and 75/5 bound	Proved
C3	$i(G) \leq 16$	Explicit 16-vertex witness	Proved
C4	$i(G) > 15$	<code>ids_le15.tree.gz</code> and independent checker	Certificate checked
C5	$i(G) = 16$ by formula route	Theorem 1 and all 3^{15} partial assignments	Proved and exhaustively instantiated
C6	The regular-graph conjecture is false	C2-C4	Established
C7	$i(G(F)) = k + \beta(F)$	Manuscript proof	Proved; novelty bounded
C8	Exact signed twice-occurrence proper 3-CNF gives cubic graph and $\mu^* = k$	Manuscript proof	Proved
C9	No cubic DIM counterexample below order 50	Definition transfer plus external 20-clause result	Conditional external dependency

The machine-readable form is `CLAIMS.json`.

4. Theorem-critical replay

Run:

```
./run_core_verification.sh
```

The script performs, in order:

1. cross-format graph and witness checks;
2. complete truth-table and minimal-unsatisfiability checks;
3. direct dependency-free graph verification and bilateral enumeration;
4. proof-tree verification from the raw graph;
5. deterministic certificate regeneration and byte comparison;
6. four targeted checker mutation tests;
7. metadata and assurance-scope checks;
8. manifest verification.

The core replay has no third-party Python dependency. It compiles the C++ programs with

```
-std=c++20 -O3 -Wall -Wextra -Wconversion -Wshadow -pedantic
```

4.1 Recorded exact outputs

Check	Result
Vertices / edges	50 / 75
Degree sequence	3^{50}
Independent dominating witness	16 vertices
Complete assignments	$2^{15} = 32,768$
Satisfying assignments	0
Minimum unsatisfied clauses	1
Assignments attaining minimum	3,318
Single-clause deletions satisfiable	20 of 20
Partial assignments	$3^{15} = 14,348,907$
Bilateral assignments	939,975

Check	Result
Minimum bilateral deficiency	1
Proof-tree nodes	256,714
Branch nodes / bound leaves	116,229 / 140,485
Maximum proof-tree depth	15
Compressed certificate size	21,803 bytes
Targeted corruptions rejected	4 of 4
Certificate regeneration	Byte-identical

The accounting check

$$116,229 + 140,485 = 256,714$$

closes exactly.

5. Optional corroborative audit

Run:

```
./run_optional_audits.sh
```

This requires the hash-locked NumPy and SciPy wheels in `environment/requirements-milp.lock`. The generic mixed-integer formulations return

- independent domination optimum 16;
- minimum maximal matching optimum 15;
- zero reported optimality gap.

These optimiser results are corroborative. The proof remains complete without SciPy or HiGHS.

6. Proof trust and sensitivity audit

6.1 Matching invariant

The upper witness is a 15-edge maximal matching. A matching edge in a cubic graph dominates at most five graph edges. Since the graph has 75 edges,

$$\mu^*(G) \geq \left\lceil \frac{75}{5} \right\rceil = 15.$$

This proof is insensitive to the formula representation.

6.2 Independent-domination upper bound

The canonical witness

$$\{0, 3, 5, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 47\}$$

is checked directly on the raw graph. It is also derived from a complete assignment that falsifies only C_{18} .

6.3 Lower-bound route A

The proof-tree checker reconstructs every proof state and branch from the raw edge list. It trusts Python parsing, integer operations and ordinary machine execution. The generator is not part of the checking trust base once the certificate exists, although byte-identical regeneration tests determinism.

The four mutations cover invalid bound, invalid branch, truncation and trailing data. They do not exhaust the checker input space.

6.4 Lower-bound route B

The second route trusts Theorem 1’s mathematical proof and an independently implemented C++ enumeration over all 3^{15} partial assignments. It does not depend on the proof-tree parser or certificate format.

The two exact routes share the canonical graph/formula but otherwise use different state representations. Their agreement reduces implementation-specific risk.

7. External dependency boundary

The central disproof is self-contained relative to the bundle. The order-50 theorem for cubic graphs with a dominating induced matching depends on Zhang, Peitl and Szeider’s result that an unsatisfiable $(3, 2, 2)$ -formula needs at least 20 clauses (Zhang et al., 2024).

THEOREM_DEPENDENCY.md verifies that the derived endpoint formula, after satisfiability-preserving deletion of tautologies and duplicate clauses, has:

1. exactly three distinct literals per retained clause;
2. no complementary pair;
3. at most two positive and two negative occurrences per variable.

EXTERNAL_DEPENDENCIES.json records DOI 10.4230/LIPIcs.SAT.2024.31, URN urn:nbn:de:0030-drops-205531, publication version and access date. It also pins the official Zenodo computational supplement supplementary_material_smallest_k_CNF.zip by repository-supplied MD5 a98d3544d736ccf3f23b9922100d468e. The official paper PDF could not be copied into the build runtime, so no local PDF SHA-256 is claimed.

8. Prior art and novelty boundary

The graph architecture has direct antecedents, especially Zhang, Peitl and Szeider’s clause-literal graph (Zhang et al., 2024). The exact twice-positive/twice-negative formula class is central to Ahadi and Dehghan (Ahadi & Dehghan, 2019). Zverovich reports a close satgraph relationship between satisfiability and independent domination (Zverovich, 2006).

The release’s bounded novelty statement is:

We prove the exact bilateral-deficiency identity; our targeted search did not locate an equivalent published statement.

No novelty is claimed for the basic graph transformation or formula class. A complete full-text satgraph comparison and specialist database search remain outstanding. PRIOR_ART_COMPARISON.md records the inspected distinctions.

9. Exact assurance matrix

Dimension	Exact v4 state	Qualification
Internal deterministic replay	Passed	Recorded environment and clean extracted copy
Independent rerun	No	No exact-v4 external execution
Independent reimplementation	No	Fable concerns a predecessor
Formal verification	No	No proof assistant
Peer review	No	No conventional specialist or journal process
Environment definition	Pinned	Digest-pinned base and hash-locked optional wheels
Container execution	Pending	Runtime unavailable during build
External v3 technical review	Completed	Major revisions; disposition recorded

The permissible release wording is “all bundled theorem-critical deterministic checks passed in the recorded environment”. Generic “verified”, “independently reproduced” or “peer reviewed” labels are not authorised.

10. Environment and clean replay

The recorded successful environment is Debian GNU/Linux 13.3, Python 3.13.5 and GNU C++ 14.2.0. The optional audit used NumPy 2.3.5 and SciPy 1.17.0. Locale, timezone, compiler flags and deterministic variables are recorded in `ENVIRONMENT.md`.

`environment/Containerfile` pins the Debian 13.3 slim base by image digest and requests exact package versions. `environment/requirements-milp.lock` pins optional wheel bytes by SHA-256. Docker/Podman was unavailable, so the container itself was not built. This is why environment reproducibility is recorded as partial rather than complete.

11. Provenance and review lineage

The same labelled graph was public in commit `7810f30c5e9da0af9ece566bdffa5844d6b34f1b` before this release ([Public counterexample repository](#), 2026). The audited source archive is preserved as `source/newproof.zip`.

Fable’s independent harness concerns `v2-theorem`, SHA-256 `106787...a589`. The supplied full review concerns exact `v3`, SHA-256 `94fe632...3e8f`, and recommended release hardening rather than rejection. Neither record is an exact-`v4` independent rerun. `VERSION_HISTORY.md`, `PROVENANCE.md` and `REVIEW_RESPONSE.md` preserve this lineage.

12. Missingness audit and limitations

Not established:

- global order-50 minimality among all cubic or regular counterexamples;
- uniqueness at order 50;
- secured priority for Theorem 1;
- exact-`v4` independent reproduction;
- formal verification;
- specialist peer review;
- successful container or continuous-integration replay;
- a content hash for the consulted external SAT paper;
- exhaustive specialist-database literature coverage.

These limitations bound the structural, novelty and assurance claims. They do not create a known defect in the explicit graph counterexample.

References

- Ahadi, A., & Dehghan, A. (2019). (2/2/3)-SAT problem and its applications in dominating set problems. *Discrete Mathematics & Theoretical Computer Science*, 21(4). <https://doi.org/10.23638/DMTCS-21-4-9>
- Public counterexample repository. (2026). *TxGraffiti conjecture 15/3 counterexample bundle*. <https://github.com/djma/TxGraffiti-conjecture3-counterexample>
- Zhang, T., Peitl, T., & Szeider, S. (2024). Small unsatisfiable k -CNFs with bounded literal occurrence. *27th International Conference on Theory and Applications of Satisfiability Testing (SAT 2024)*, 305, 31:1–31:22. <https://doi.org/10.4230/LIPIcs.SAT.2024.31>
- Zverovich, I. E. (2006). Satgraphs and independent domination. Part 1. *Theoretical Computer Science*, 352(1–3), 47–56. <https://doi.org/10.1016/j.tcs.2005.08.038>